CRSS-Python v1.0.0
A Full-Lifecycle Safety Standard for Python

CRSS = Certifiable Reliability & Safety Standard for Python

Table des matières

1. Executive Summary

CRSS-Python v1.0.0 represents a landmark achievement in the world of Python safety engineering. For the first time, a comprehensive, certification-grade standard has been developed to enable Python to operate in supervisory safety contexts across industries such as automotive, industrial automation, medical technologies, and avionics. This release formalizes the standard as a mature, governed, and publicly available framework.
It is a full-Lifecycle Safety Standard for Python, defining a complete, certifiable safety framework tailored to modern Python ecosystems. It addresses not only coding style, but also determinism, criticality, deployment integrity, toolchain confidence, configuration governance, and certification evidence.

CRSS stands for Certifiable Reliability & Safety Standard for Python. The standard introduces two profiles (Core and Strict), three safety levels (A/B/C), and a critical / non-critical phase model that ensures safety-critical logic is deterministic, bounded, allocation-free, and free from I/O and blocking behavior.
Where traditional guidelines (such as PEP8 or style checkers) focus on readability, CRSS-Python targets high-integrity software. It integrates: Core and Strict coding rules Safety levels and criticality propagation Deterministic execution constraints for Strict-Level-A Import and inheritance safety policies Big data, caching, microservices & networking robustness rules Memory and GC determinism Regex ReDoS protections Coverage, MC/DC, and robustness test requirements Deployment immutability and baseline management SCEM (Safety Case Evidence Model) and compliance processes Toolchain Confidence Assessment (TCA) Rollback, hotfix, and drift-prevention policies

CRSS-Python does not attempt to turn Python into a hard real-time RTOS language. Instead, it defines the strongest achievable safety discipline on CPython by restricting what can happen in critical paths, isolating non-critical behavior, and wrapping everything in a rigorous deployment, evidence, and governance model.

2. Key Achievements

✓ Unified Safety Model
✓ Deterministic Execution Guarantees for Critical Code
✓ Full Core & Strict Profile Specifications
✓ Safety Levels, Modes, and Propagation Rules
✓ Deployment, Baseline, and Zero-Drift Policies
✓ Compliance Master and Certification Framework
✓ SCEM Evidence and Maturity Model
✓ Tooling & Automation Governance
✓ Full Rule Catalog with Identifiers

3. Industry Impact

Before CRSS-Python, the use of Python in high-integrity environments was typically discouraged due to lack of deterministic behavior, weak deployment controls, and absence of formal compliance tooling. CRSS-Python changes this landscape by providing a certification-ready safety framework and a governance model that aligns with ASIL D, SIL 3, and comparable supervisory safety levels.

When fully applied, CRSS-Python supports supervisory roles at the highest integrity levels, including ASIL-D supervisory, SIL-3 supervisory, IEC 62304 Class C supporting systems, DO-278A ground systems, and high-assurance industrial automation or mission-critical utilities. Supervisory components can be implemented in Python with confidence, while primary actuation loops remain in traditional RTOS / certified language environments.

4. Technical Foundations of CRSS-Python

4.1 Overview

CRSS-Python defines a deterministic, traceable, full-lifecycle safety ecosystem for Python. It goes far beyond coding rules and introduces structured controls across architecture, runtime semantics, deployment, and certification evidence.

4.2 Core and Strict Profiles

CRSS defines two rule profiles:

- Core Profile: General safety-oriented rules suitable for non-critical and moderately critical systems.
- Strict Profile: High-integrity rule set for supervisory safety-critical applications.

Strict Profile includes tighter guarantees regarding determinism, object lifecycle, GC control, allocation discipline, concurrency, I/O, and environmental stability.

4.3 Safety Levels (A/B/C)

CRSS introduces three internal safety levels:

- Level A Highest criticality, deterministic, bounded, allocation-free critical execution.
- Level B: Important safety logic with moderate constraints.
- Level C: Non-critical application logic.

Safety levels propagate structurally through import boundaries and inheritance according to MAR (Mode Assignment Register).

4.4 Critical and Non-Critical Phase Model

CRSS separates execution into:

- @critical: deterministic, bounded execution with no I/O, blocking, or dynamic allocation.
- @non_critical_phase: allowed to perform I/O, caching, allocations, parsing, configuration, etc.

Strict MUST-NOT rules remain enforced everywhere. The distinction applies only to operational permissions.

4.5 Safety-Level Propagation Mode

If Level A code imports or inherits from another module/class, the imported element is elevated to Level A for verification. This ensures deterministic safety boundaries. Level A → Level B calls are allowed, but Level B → Level A requires review.

4.6 Determinism Constraints

Strict-A requires:

- no GC activity during @critical
- no unbounded data structures
- no blocking operations
- no time-based logic in decision loops
- bounded execution path (no recursion, no dynamic dispatch)

These constraints mirror requirements in ISO 26262, IEC 61508, and DO-178C determinism objectives

5. The CRSS-Python Safety Knowledge Base

5.1 Purpose of the Knowledge Base

The CRSS-Python Safety Knowledge Base consolidates all rules, architectural constraints, testing requirements, deployment controls, safety semantics, and evidence-generation processes into a single, coherent framework. It enables certifiable, auditable, and reproducible safety behavior across the full software lifecycle.

5.2 Components of the Knowledge Base

The Knowledge Base integrates multiple domains of safety engineering, including:

- Coding standards (Core / Strict Rulesets)
- Safety Levels A/B/C and propagation model
- Critical & non-critical phase semantics
- Import and inheritance safety restrictions
- Architectural layering and module-boundary rules
- Deployment, baseline, and configuration integrity
- Toolchain Confidence Assessment (TCA)
- Testing, coverage, MC/DC, and robustness requirements
- SCEM: Safety Case Evidence Model
- Logging, diagnostics, and runtime observability
- Networking, caching, big-data, and microservices rules
- Sensitive data handling and security constraints
- Numerical determinism and Regex ReDoS protections
- Rollback, failover, and update-governance model

5.3 Integration and Traceability

Every CRSS requirement is traceable:

- upward → to safety goals, architectural decisions, and SIL/ASIL requirements • downward → to specific rules, tests, design constraints, and runtime behaviorThrough SCEM, baseline manifests, and acceptance rules, each project can generate a complete chain of objective evidence for audit and certification.

5.4 Profiles + Safety Levels + Phases = Unified Semantics

CRSS combines:

- Profile (Core / Strict)
- Safety Level (A/B/C)
- Phase(critical/non-critical)

into a unified compliance model that allows tools to reason accurately about:

- what is allowed
- what must be bounded
- what must be deterministic
- where I/O is forbidden
- where allocation is forbidden
- where concurrency is forbidden

This enables Python code to participate in high-integrity systems while maintaining developer productivity outside critical sections.

5.5 Deterministic Execution as a First-Class Safety Property

Strict-Level-A @critical execution is defined to be:

- Deterministic
- Bounded
- Reproducible
- allocation-free
- I/O-free
- free of timing and blocking variability

This determinism foundation is essential to supervisory safety roles in ASIL-D, SIL-3, and similar domains.

5.6 Architectural Guardrails

The Knowledge Base establishes a robust architecture framework:

- Strict one-way import layering
- Inheritance depth limitation
- Safety-level propagation rules
- "No downward leakage" from non-critical to critical
- Clear separation between decision logic and operational logic
- Encapsulation of side-effects in non-critical phases

5.7 Tooling and Automation Support

CRSS includes formal specifications for:

- linting and static rule enforcement
- dynamic checks for critical phases
- coverage measurement (branch, MC/DC)
- SCEM generation
- rule-scoped configuration loaders
- drift detection and baseline validation

This ensures the standard is not only readable, it is executable.

6. CRSS Safety Ecosystem vs Traditional Coding Guidelines

6.1 Not Just a Style Guide

Traditional language rulebooks (e.g., PEP8, Google Style Guide, or generic linters) focus primarily on readability, formatting, naming, and conventional good practice. These are valuable, but insufficient for safety-critical software.

CRSS-Python is not a style guide, it is a complete safety ecosystem. It governs execution determinism, architecture, runtime behavior, development processes, deployment reproducibility, testing rigor, and certification evidence. It is comparable in scope to ISO 26262 Part 6 + Part 8 + MISRA + DO-178C structures combined.

6.2 Beyond MISRA-Style Rulebooks

CRSS-Python includes coding rules like MISRA, but extends far beyond them.

MISRA C focuses exclusively on source-level constraints. CRSS additionally includes:

- determinism guarantees
- safety-level propagation
- critical phase semantics
- constraints on I/O, allocation, GC, and scheduling
- architectural boundaries for imports and inheritance
- deployment integrity rules
- TCA (Toolchain Confidence Assessment)
- SCEM Safety Case Evidence Model
- rollback and drift-prevention rules
- cache/big-data/microservices/networking robustness

CRSS is closer to a full functional safety process standard than a simple coding rule set.

6.3 Lifecycle Governance vs Code-Only Governance

PEP8 and style guides govern code formatting.

CRSS governs:

Design, architecture, execution, runtime behavior, deployment, test evidence, baselines, compliance acceptance, certification artifact generation

This ensures traceability from safety goals → architecture → rules → tests → deployment → operation.

6.4 Deterministic Criticality is Unique to CRSS

No general Python guideline enforces:

- GC disablement
- allocation-free critical phases
- I/O-free critical execution
- deterministic bounded execution
- safety-level propagation
- structural isolation between critical and non-critical

CRSS introduces these guarantees specifically to make Python suitable for ASIL-D/SIL-3 supervisory domains.

6.5 Architecture & Structural Integrity

Standard coding guidelines rarely control:

- import direction
- module-layer hierarchy
- inheritance depth and safety elevation
- component boundaries
- side-effect isolation
- safety-level isolation

CRSS makes these architectural concerns first-class safety properties.

6.6 Deployment & Configuration Integrity

Traditional guidelines do not cover:

- reproducible builds
- immutable deployments
- version-locked baselines
- drift detection
- rollback safety
- dependency repository isolation
- exact environment reconstruction

CRSS ensures that code validated in testing is exactly the same code running in production — bit-for-bit and dependency-for-dependency.

6.7 Certification Alignment

CRSS aligns structurally with:
ISO 26262, IEC 61508, IEC 62304 DO-278A, DO-178C (non-actuating roles)
Because it includes: documented lifecycle, configuration control, determinism model, evidence model (SCEM), TCA requirements, acceptance rules, traceability and auditability

This positions CRSS as an unprecedented safety standard in the Python ecosystem.

7. Full Lifecycle Scope

7.1 Overview of the CRSS Lifecycle

CRSS-Python establishes a unified lifecycle that begins at system conception and ends at long-term operation and maintenance. Unlike typical coding standards that focus only on development activities, CRSS defines requirements across:

- Requirements engineering
- Architecture and design
- Implementation
- Verification and validation
- Deployment and release governance
- Runtime observability
- Change management
- Certification evidence generation
- Long-term sustainability

7.2 Requirements → Architecture → Rules Flowdown

Safety goals and system-level requirements flow down directly into architectural decisions following:

1. Functional Safety Requirements 2. System Architecture Safety Constraints 3. CRSS Coding Rules 4. Testing & Verification Requirements 5. Deployment & Operational Integrity Requirements

This ensures that every requirement has:

- upstream rationale
- downstream enforcement
- measurable verification procedures

7.3 Development Phase Controls

CRSS development requirements include:

- conformance to Core or Strict profiles
- correct safety-level assignment (A/B/C)
- application of import and inheritance constraints
- bounded data structures
- deterministic critical regions
- strict separation of critical and non-critical phases
- mandatory documentation for architectural decisions

7.4 Verification & Testing Phase

CRSS mandates structured testing activities:

- unit tests
- integration tests
- security tests
- fault-injection tests
- robustness and stress tests
- MC/DC and branch coverage (Strict-A)
- timing expectation tests (non-hard real time)
- data-boundary and input domain tests
- negative testing for malformed input

All tests contribute to the SCEM (Safety Case Evidence Model).

7.5 Deployment & Release Management Phase

Deployment integrity is a pillar of CRSS. It requires:

- immutability of deployed artifacts
- exact baseline matching (bit-for-bit determinism)
- reproducible build environments
- frozen dependency repositories
- versioned and traceable OS, Python interpreter, and containers
- a formal release certificate linked to a Configuration Baseline Manifest (CBM)

Any divergence between test and production invalidates compliance and requires reapproval.

7.6 Runtime Operation Phase

CRSS runtime expectations include:

- watchdog supervision of critical services
- robust logging and monitoring
- enforcement of safety constraints at runtime (e.g., GC disabled during @critical)
- drift detection for configuration, environment, and dependencies
- isolation between critical and non-critical components
- stable service boundaries in microservice architectures

7.7 Maintenance & Change Management Phase

CRSS prohibits uncontrolled updates. Every change must follow:

1. Change Proposal
2. Safety Impact Assessment
3. Full or partial SCEM regeneration
4. Re-testing under updated baseline
5. Approval and release certification

Hotfixes follow a limited but still controlled variant of this workflow and must never bypass safety validation.

7.8 Certification & Evidence Model Integration

CRSS provides a certifier-ready structure:

- SCEM
- MAR (Mode Assignment Register)
- CBM (Configuration Baseline Manifest)
- Acceptance Rules Addendum
- Compliance Report
- Toolchain Confidence Assessment

These support auditability for ASIL-D supervisory systems, SIL-3 systems, Class C medical systems, and mission-critical ground systems.

7.9 Long-Term Sustainability

CRSS ensures sustainability through:

- version governance
- reproducibility of builds even years later
- private dependency mirroring
- strict change control
- structured refactoring paths
- continuous validation of safety assumptions

This ensures safe operation across the full lifecycle of long-lived industrial and mission-critical systems.

8. Determinism & Criticality Model

8.1 Determinism as a Safety Property

In CRSS-Python, determinism is not a performance enhancement—it is a safety requirement. Deterministic behavior ensures that a safety-critical execution path produces the same results under the same conditions. This principle is essential for ASIL-D supervisory elements, SIL-3 systems, and any context where predictability directly impacts safety.

The CRSS determinism model eliminates timing nondeterminism, scheduling variability, GC pauses, unbounded memory operations, and unpredictable side effects inside critical execution.

8.2 The Three-Layer Execution Model

CRSS enforces a structured execution model:

- Profile (Core / Strict): sets the global rule constraints
- Safety Level (A/B/C): sets verification and testing rigor
- Phase (critical / non-critical): controls what operations are permitted

This layered separation ensures that critical logic behaves deterministically even when surrounding application logic remains flexible.

8.3 Strict-Level-A Critical Execution Requirements

Strict-Level-A is the highest safety classification in CRSS for Python.
Every @critical call executed under Strict-Level-A MUST comply with:

- deterministic, bounded execution
- no allocation (no new objects, no resizing structures)
- no GC activity; GC must be disabled
- no file, network, database, or IPC I/O
- no blocking (no sleep, no socket waits, no joins, no locks)
- no time-based logic (no reliance on wall-clock scheduling)
- no dynamic dispatch, recursion, or metaprogramming
- no calls to non-critical code

Critical execution must be pure computation using pre-validated, pre-allocated, and pre-bounded structures.

8.4 Non-Critical Phase Semantics

Non-critical phases, including functions marked @non_critical_phase may perform operational tasks such as:

- I/O
- file access
- network communication
- object creation and allocation
- caching and data loading
- configuration and parsing

However:

- all Strict MUST-NOT rules remain enforced
- safety-level constraints still apply
- non-critical logic may not contaminate critical logic
- non-critical behavior may not be called from @critical sections
- non-critical operations must not introduce instability or nondeterministic effects that propagate into critical code

8.5 Critical Entry & Exit Model

Criticality is defined per function call, not per file. Entering a @critical function activates:

- GC-disabled mode
- determinism constraints
- allocation bans
- I/O bans
- Exiting a @critical function re-enables the non-critical execution environment

This model allows developers to isolate small, deterministic decision loops inside a larger operational system.

8.6 Safety-Level Propagation and Determinism

- Safety levels (A/B/C) propagate upward through imports and inheritance.
- If Level-A critical code depends on another component (via import or inheritance), that component becomes Level-A for verification and audit purposes.

This ensures deterministic boundaries:

- Level-A → may call Level-B
- Level-A → may NOT depend on Level-C
- Level-B → calling Level-A is allowed
- Level-C → must never inject behavior into Level-A code paths

This guarantees the determinism of top-level control logic.

8.7 GC, Memory Discipline & Object Lifecycle

CRSS enforces strict memory semantics for deterministic execution:

- GC must be disabled in @critical
- no allocation inside @critical
- object graphs must be acyclic

- collections must be prebounded
- caches must have fixed maximum size
- memory lifecycle must be explicit

These constraints eliminate Python's most unpredictable runtime behaviors.

8.8 Timing and Scheduling Discipline

CRSS prohibits:

- sleep() or time-based loops
- reliance on CPU scheduling quantum
- I/O waits inside critical logic
- spinning or polling loops that depend on CPU speed

While CRSS does not claim hard real-time determinism (CPython cannot achieve this), it ensures consistent bounded behavior suitable for supervisory-grade safety.

8.9 Summary of Determinism Guarantees

CRSS guarantees that:

- critical logic is deterministic
- side effects are contained
- memory is bounded
- runtime behavior is predictable
- GC and allocation are controlled
- safety levels propagate correctly
- nondeterministic features are isolated

This allows Python to serve as a safe supervisory layer in functional safety ecosystems.

9 Architecture & Structural Safety

9.7 Architectural Safety Principles

CRSS-Python enforces structural safety as a first-class property. Architectural safety ensures that code organization, imports, inheritance, module boundaries, and component interactions cannot undermine critical safety guarantees. Architecture must be stable, hierarchical, traceable, and free from hidden coupling or nondeterministic flows.

9.8 Import Layering Rules

CRSS defines a strict, one-directional layering model:

• High-criticality modules may NOT depend on lower-integrity modules unless promoted via MAR• Low-criticality modules may depend on higher-criticality modules • Cyclic imports are forbidden• Critical code may only import other critical-safe modules • Non-critical modules must never leak I/O or state into critical paths

This protects critical logic from contamination and nondeterministic interactions.

9.9 Inheritance Safety Model

CRSS enforces controlled inheritance:

• Only single-level inheritance is allowed (depth of 1) • Inheriting from a Level-A class elevates the subclass to Level-A • Multiple inheritance is forbidden • Inheritance chains may not introduce nondeterministic behavior • Base classes for critical logic must be stable, well-documented, and free of side-effects

This prevents untraceable behavior that often arises from Python's flexible OOP mechanisms.

9.10 Safety-Level Propagation Across Architecture

Safety levels propagate across architecture as follows:

• If module M_A (Level A) imports module M_B (Level B), then M_B becomes Level A for verification • Classes inherit the stricter safety level of their ancestors • Functions imported into Level A contexts must be verified as Level A

This ensures structural integrity even when components originate at different safety levels.

9.11 Component Boundary Isolation

CRSS defines strict boundaries between system components:

- Decision logic (critical) is isolated from operational logic (non-critical)
- Safety-critical computation must not cross module boundaries that include I/O
- Shared state is forbidden; explicit data transfer interfaces must be used
- All communication must be validated, bounded, and deterministic

This minimizes coupling and enhances traceability for certification.

9.12 Microservice Safety Design

For distributed systems, CRSS requires:

- deterministic, schema-bound API interactions
- retries with bounded backoff
- validated request/response payloads
- network-failure-tolerant design
- service timeouts and circuit breakers
- no critical logic distributed across network calls

Critical safety decisions MUST occur locally to guarantee determinism.

9.13 Safe Dataflow Architecture (Diagram)

---------- Non-Critical I/O Layer---------- • Input adapters • Network clients • Device interfaces • File/db access

↓ sanitized data

---------- Validation & Preprocessing----------• schema validation • bounds checking • sanitization • safe parsing

↓ bounded, validated data

---------- Critical Decision Layer ----------• deterministic computation • allocation-free logic • GC disabled • pure functions

↓ decisions

---------- Non-Critical Output Layer ---------- • logging • networking • publishing • storage

9.14 Avoiding Hidden Coupling

CRSS prohibits:

- global mutable state
- singletons
- implicit initialization logic
- thread-local state affecting critical logic
- side-effects during import
- magic dynamic dispatch

All behavior must be explicit, traceable, and reviewable.

9.15 Architectural Review Requirements

All CRSS projects must conduct formal architectural reviews, including:

- dependency graphs
- module boundary audits
- safety propagation mapping
- inheritance chain validation
- criticality boundary definition
- import-layer compliance
- I/O isolation validation

Architecture must be stable and documented for certification.

9.16 Summary of Structural Safety Guarantees

CRSS delivers:

- clean layering
- deterministic dataflow
- bounded interactions
- tightly controlled imports
- safe inheritance
- explicit component boundaries
- isolation of critical decision logic

These architectural guarantees form the backbone of certifiable Python systems.

10 Memory, GC, and Runtime Behavior

10.7 Deterministic Memory as a Safety Property

Memory nondeterminism is one of the largest risks in Python safety-critical systems. CRSS enforces strict policies that ensure the memory footprint and lifetime of objects are fully predictable during critical execution.

10.8 GC Control Model

CRSS defines a two-phase GC model:

- Non-critical phases: GC enabled, allocations allowed
- Critical phases: GC MUST be disabled

Critical functions must operate without triggering any GC cycle. All objects used inside critical code must be pre-allocated or come from safe, bounded pools.

10.9 Allocation Policy by Phase

CRSS Phase Allocation Rules:
Non-Critical Phase:

- Allocation allowed
- Caching allowed (bounded)
- Object pools may be refilled
- No unbounded growth

Critical Phase:

- No dynamic allocation
- No structure resizing
- No implicit conversions that allocate
- No string concatenation creating new objects

10.10 Bounded Collections

All collections used in critical execution must be:

• Sized deterministically • Capacity fixed before entering a critical phase • Never appended, extended, resized, or rehashed during critical execution

This ensures predictable memory footprint and eliminates hash-table resizing time spikes.

10.11 Object Graph Constraints

Object graphs used by critical logic must be acyclic. Cyclic references risk:

- GC pauses
- unpredictable reclamation behavior
- memory retention beyond intended lifetime No string concatenation creating new objects

CRSS enforces structural acyclicity via static analysis and manual review.

10.12 Safe Caching and Eviction

Caching is permitted only when:

- Per-key and overall cache size is bounded
- Eviction is deterministic
- Cache is not accessed during critical logic unless read-only

Unbounded or lazy caches are prohibited.

10.13 Resource Lifetime Commitments

All resources used by critical code must have:

- deterministic lifetime
- deterministic cleanup
- no external dependency for disposal Files, sockets, and remote handles are prohibited inside critical regions
- unpredictable reclamation behavior

10.14 Interpreter-Level Considerations

Python's interpreter introduces sources of nondeterminism:

- reference-count behavior
- dictionary key hashing
- dynamic attribute lookup
- C-extension memory behavior

CRSS mitigates these by:

- forbidding mutable dicts in critical logic
- requiring pre-binding of all attributes
- banning C-extensions unless verified deterministic

10.15 Memory Safety Review Checklist

A CRSS project must validate:

- GC is disabled in all critical functions
- Collections have fixed size
- No dynamic allocation inside critical logic
- No hidden allocations via string formatting, f-strings, exceptions
- Object pools documented and bounded
- No cyclic object graphs

10.16 Memory Safety Summary

Strict memory governance is one of the key reasons CRSS-Python can reach ASIL-D supervisory and SIL-3 supervisory levels. Deterministic, bounded, and allocation-free critical execution is essential for certifiable safety behavior.

11 Network, Microservices & Distributed Safety

11.7 Distributed Systems as a Safety Risk

Distributed systems introduce nondeterminism from latency, bandwidth, network jitter, packet loss, remote timeouts, and external system failures. CRSS-Python treats all network interactions as non-deterministic by default and strictly forbids placing critical safety logic across network boundaries.

11.8 Deterministic Network Interaction Model

CRSS enforces:

- Network communication MUST NOT occur inside critical phases
- All network I/O must be bounded by explicit timeouts
- Retry mechanisms must use capped exponential backoff
- No unbounded waiting or blocking on remote calls
- All inputs from remote systems must undergo validation

This ensures network behavior can never jeopardize deterministic safety computation.

11.9 Microservice Safety Design

CRSS microservice safety rules:

- Critical decisions MUST be made locally
- Remote calls may only influence non-critical preprocessing
- All remote APIs must use fixed schemas
- Validation is mandatory before data enters any safety-relevant component
- No implicit state sharing or reliance on remote freshness guarantees

This prevents safety-critical logic from depending on the reliability of networked systems.

11.10 Safe API Schemas

All remote API inputs must:

- be schema validated (JSON-Schema, protobuf, etc.)
- enforce strict types
- enforce bounded size
- reject unknown fields
- define deterministic fallbacks for missing/invalid data

The safety of the system is directly tied to the strictness of data schemas.

11.11 Handling Partial Failures

Distributed systems fail partially and asymmetrically. CRSS enforces:

- fail-safe defaults on remote timeouts
- circuit breakers for unstable services
- no infinite retries
- fallback decision logic (local, deterministic)
- strict separation of degraded vs normal modes

Safety relies on predictable, bounded error behavior.

11.12 Deterministic Distributed Architecture (Diagram)

---------- Non-Critical Network Layer ---------- • HTTP/GRPC clients • SFTP/FTP • Message brokers • Socket interfaces

↓ validated and bounded

---------- Sanitization & Preprocessing ----------• schema validation • size constraints • sanitization • transformation

↓ safe data

---------- Critical Decision Layer ----------• no network • deterministic logic • pure computation

↓ decision

---------- Non-Critical Output Layer ---------- • publish result • send API response • store to DB

11.13 Timeout, Retry & Backoff Safety Rules

CRSS requires:

- fixed maximum timeout
- bounded retry count
- exponential backoff with ceiling
- fallback error paths
- logging of remote instability

This ensures no distributed behavior can cause uncontrolled delays.

11.14 Isolation of Safety-Relevant Logic

Safety logic must always be:

- Local
- Deterministic
- independent from external timing
- independent from the state or availability of remote systems

Only non-critical preparatory or follow-up processing may span distributed nodes.

11.15 Networking Safety Review Checklist

A CRSS system must verify:

- critical logic contains zero network operations
- network inputs are schema-validated
- retries and timeouts are bounded
- microservice boundaries match safety levels
- circuit breakers and fallback logic are implemented
- no safety decision depends on remote freshness

These conditions ensure distributed safety integrity.

11.16 Summary of Distributed Safety Constraints

CRSS delivers a deterministic, bounded, and certifiable model for microservice and network-heavy architectures. Critical logic remains fully isolated from remote failure modes, enabling ASIL-D and SIL-3 supervisory safety behavior even in modern distributed software ecosystems.

12 Security, Sensitive Data & Hardening

12.7 Security as a Safety Enabler

In CRSS-Python, safety and security are inseparable. A data breach, corrupted configuration, unauthorized access, or tampered dependency can directly compromise safety-critical correctness. CRSS therefore treats security not as an add-on but as a mandatory prerequisite for safety integrity.

12.8 Threat Model Alignment

CRSS enforces a unified threat model:

- Safety hazards (wrong output, misbehavior)
- Security threats (tampering, injection, unauthorized access)
- Reliability threats (unavailability, resource exhaustion)

All three must be mitigated concurrently in any component affecting safety behavior. The threat model focuses on protecting deterministic execution, validated data flow, and system integrity.

12.9 Secure Input Handling

All external inputs must be:

- Sanitized
- schema-validated
- size-bounded
- type-enforced
- encoding-normalized
- rejected on invalidity

No direct JSON, CSV, or YAML loading without validation is permitted. Ambiguous or multi-encoding data must be normalized to UTF-8 and sanitized.

12.10 Sensitive Data Handling

CRSS defines strict rules for sensitive data:

- passwords, keys, tokens, secrets must NEVER appear in logs
- sensitive data must be redacted or hashed before persistence
- secrets must not be hardcoded
- environment variables must be validated and access-controlled
- configuration files containing secrets must be encrypted or access-restricted

Sensitive data leakage is a safety hazard because it can enable unauthorized system manipulation.

12.11 Secure Logging Model

Logging must follow:

- no sensitive data in logs
- no raw request payloads containing secrets
- redaction filters enforced by policy
- deterministic log formatting
- separate log-level policies for critical vs. non-critical phases

Critical code may NOT perform logging. Logging is strictly non-critical.

12.12 Authentication & Authorization (RBAC)

CRSS enforces:

- role-based access control for administrative operations
- principle of least privilege
- explicit allowlists for privileged actions
- no dynamic privilege escalation

All access to configuration, deployment systems, or safety-significant components must be governed by RBAC.

12.13 Cryptographic Boundaries

CRSS defines clear cryptographic rules:

- only approved, industry-grade algorithms
- no custom crypto
- keys must be rotated
- certificates must be validated
- TLS used for all remote communication
- in-memory secrets must be zeroized when no longer needed

Crypto misuse is treated as a safety and security violation.

12.14 Injection & Code Execution Protection

The following are strictly forbidden:

- eval / exec
- dynamic imports involving untrusted strings
- shell command construction from variable input
- SQL queries without parameterization
- deserialization of untrusted objects

CRSS enforces that no untrusted data path may reach dynamic execution points.

12.15 Supply-Chain Integrity

All third-party components must:

- come from pinned, mirrored repositories
- be scanned and verified
- include SBOM entries
- undergo dependency risk assessment

No runtime installation of packages is allowed. Supply-chain integrity is fundamental for certification.

12.16 Safety-Security Interaction Model

Security hardening directly enhances safety by:

- preventing system corruption
- ensuring determinism
- blocking unauthorized behavior
- protecting integrity of critical logic
- safeguarding configuration and deployment artifacts

CRSS treats security controls as structural safety requirements.

12.17 Security Review Checklist

CRSS security verification includes:

- input validation coverage
- secret-handling review
- logging inspection
- RBAC mapping
- dependency integrity audit
- crypto boundary review
- threat modeling completeness

Projects must fulfill all checklist items before certification.

12.18 Summary of Security Requirements

CRSS incorporates a deep, multi-layered security model tightly integrated into the overall safety lifecycle. By defining strict controls for input validation, secret handling, cryptography, access control, logging, and supply-chain integrity, CRSS ensures that security incidents cannot compromise deterministic and certifiable safety behavior.

13 Testing, Coverage & Verification

13.7 Testing as a Safety Requirement

CRSS-Python treats testing not as a development practice but as a core **safety requirement**. Testing provides the objective evidence necessary for certification, demonstrating that safety goals, deterministic execution constraints, and behavioral guarantees are met.

13.8 Multi-Layered Test Strategy

CRSS mandates a comprehensive, multi-tier test strategy:

- Unit Tests
- Integration Tests
- End-to-End Functional Tests
- Negative and Robustness Tests
- Fault-Injection Tests
- Performance Bounds Tests
- Determinism and Timing-Stability Tests
- Security Tests

Each layer contributes unique evidence in the safety case.

13.9 Unit Testing Requirements

All safety-relevant units must be covered by deterministic, isolated unit tests.
Requirements:

- deterministic inputs
- no network or file dependencies
- mocks for external resources
- explicit boundary and edge-case tests
- coverage evidence required (per function)

Unit tests must not rely on randomness unless seeded and repeatable.

13.10 Integration Testing Requirements

Integration tests ensure correct cooperation between modules:

- bounded test environments
- stable test infrastructure
- deterministic replay capability
- full validation of data interfaces
- strict separation from production systems

Integration failures are treated as safety hazards.

13.11 MC/DC Coverage Model for Python

CRSS applies Modified Condition / Decision Coverage (MC/DC) for Level A logic.
CRSS MC/DC Rules:

- every condition must independently influence the decision outcome
- test cases must demonstrate pairwise independence
- boolean logic must be fully exercised
- multiple branches in a single statement must be independently validated

Although Python lacks native MC/DC tools, CRSS requires:

- manual MC/DC tables
- tool-assisted boolean expansion
- automated enforcement via SCEM templates

13.12 Path Coverage Requirements

Critical logic must achieve:

- 100% function coverage
- 100% branch coverage
- 100% MC/DC coverage (Level A)

Non-critical safety-relevant units require:

- ≥ 90% branch coverage
- mandatory coverage of all safety-relevant branches
- justification for any exclusions

No critical path may remain untested.

13.13 Negative & Robustness Testing

Negative tests must cover:

- invalid inputs
- malformed data
- oversized payloads
- missing fields
- corrupted encodings
- schema violations
- network timeouts
- exception-raising behavior

Negative testing proves that the system fails safely.

13.14 Fault-Injection Testing

Fault-injection verifies robustness under disruptive conditions:

- simulated network failures
- corrupted file reads
- delayed responses
- forced exceptions
- GC pressure (non-critical only)
- artificial high-load scenarios

Fault-injection is mandatory for safety certification.

13.15 Determinism Verification

CRSS requires that timing behavior of critical code remains stable:

- repeated execution time variance must remain within certified bound
- GC must remain disabled
- no blocking calls permitted
- input-dependent timing variation must be bounded

Determinism tests ensure the system is certifiable for Level A/B safety.

13.16 Acceptance Rules for Test Completeness

A module is test-complete when:

- all required coverage targets met
- MC/DC validated
- all safety-relevant behaviors tested
- no untested branches in critical code
- fault-injection performed
- negative tests validated
- evidence included in SCEM

Failing any item results in test incompleteness and certification rejection.

13.17 SCEM: Test Evidence Requirements

SCEM requires the following evidence:

- Coverage reports
- MC/DC tables
- Test logs
- Test reproducibility report
- Fault-injection evidence
- Timing variance & determinism report
- Configuration of test infrastructure
- Commit ID, environment hash, dependency baseline

This ensures long-term reproducibility and auditability.

13.18 Summary of Verification Requirements

CRSS defines one of the most rigorous testing frameworks in the Python ecosystem. Verification requirements exceed typical industry standards and align with ASIL-D / SIL-3 supervisory expectations. Determinism, coverage completeness, robustness, and reproducibility are the pillars of CRSS verification.

14 Deployment, Release, Baseline & Configuration Integrity

14.7 Deployment Integrity as a Safety Requirement

CRSS-Python elevates deployment integrity to a primary safety requirement. A system that executes with different configurations, dependencies, interpreter versions, or infrastructure characteristics than those used during certification cannot be considered safe.

Deterministic execution requires deterministic deployment.

14.8 Immutable Deployments

Strict rules apply:

- Production deployments must be immutable
- No runtime installation of packages
- No environmental mutations
- No unbounded configuration flexibility
- No external dependency drift (PyPI, apt, OS updates)

Immutable deployments guarantee long-term reproducibility.

14.9 CConfiguration Baseline Manifest (CBM)

Every CRSS-compliant release must include a CBM containing:

- source commit hash
- interpreter version
- OS & kernel version
- all dependency versions (strictly pinned)
- build tool versions
- compile flags
- runtime environment configuration
- container/VM cryptographic hashes
- test infrastructure metadata

The CBM defines the exact environment required for certified behavior.

14.10 Reproducible Build Requirements

CRSS mandates full reproducibility:

- identical build output across environments
- bit-for-bit reproducibility where feasible
- functional equivalence guaranteed everywhere

Artifacts include:

- reproducible wheels
- reproducible Docker/OCI images
- deterministic bytecode generation

14.11 Zero-Drift Enforcement

Production systems must never drift from their certified baseline.
Zero-drift means:

- no silent package updates
- no background OS updates
- no config file changes
- no mutable environment variables
- no unreviewed hotfixes

Any drift invalidates certification and requires full re-approval.

14.12 Release Certification Requirements

A release must include:

- SCEM evidence
- Compliance Certificate
- CBM
- Release Notes
- dependency SBOM
- TCA (Toolchain Confidence Assessment)

Only baselined releases may be deployed.

14.13 Rollback Strategy

CRSS requires a provable, deterministic rollback strategy:

- previous certified version must remain deployable
- hotfixes must never block rollback
- configuration must be reversible
- data migrations must support downgrade or snapshot-restore

Rollback is a core part of safety integrity.

14.14 Change Impact Analysis

Any change requires:

- root-cause analysis
- safety impact evaluation
- dependency risk analysis
- architectural boundary review
- test impact assessment

No change may bypass structured review.

14.15 Configuration & Platform Test Matrix

All supported platforms must be tested:

- Python interpreter version
- OS version
- CPU architecture
- hardware characteristics

The test matrix must be included in SCEM and referenced in CBM.

14.16 Deployment Review Checklist

Every deployment must confirm:

- CBM hash match
- no drift
- reproducible build integrity
- correct interpreter
- correct dependency tree
- correct OS and kernel
- correct hardware class

Deployment errors directly compromise safety certification.

14.17 Summary of Deployment & Integrity Model

CRSS defines one of the most rigorous deployment governance systems in the Python ecosystem. Immutability, reproducibility, baseline control, rollback capability, and certification-driven releases ensure that Python systems can operate within ASIL-D and SIL-3 supervisory safety requirements.

15 Tooling, Automation & Toolchain Confidence Assessment (TCA)

15.7 Purpose of Tooling & Automation

CRSS-Python defines tooling and automation as integral parts of the safety lifecycle. Automation provides repeatability, objectivity, evidence capture and reduces human error, making it a necessary component for certification in high-integrity systems.

15.8 Static Analysis Requirements

CRSS requires static analysis to enforce:

- rule compliance for Core & Strict profiles
- phase-based rule enforcement (critical vs non-critical)
- safety-level propagation validation
- forbidden pattern detection (eval, dynamic imports, unsafe regex, etc.)
- dependency and import layering analysis
- complexity and maintainability scoring

Static analysis must be deterministic, auditable, and version locked.

15.9 Automated Compliance Checking

Automated compliance tools must:

- read MAR/Safety Level Assignments
- detect rule violations by profile and phase
- produce structured JSON/XML reports
- integrate into CI pipelines
- fail builds on MUST and MUST-NOT violations

No manual override is permitted for MUST/MUST-NOT violations.

15.10 Build System Requirements

The build system must:

- be deterministic
- run in clean, isolated environments
- record all build metadata into the CBM
- detect dependency drift
- enforce reproducibility via hashing

Build systems must operate without network access during build.TODO

15.11 Continuous Testing & Evidence Capture

Automation must:

- run the full test suite
- produce coverage reports
- generate MC/DC evidence (manual+tool-assisted)
- capture logs, system state, and timing variance
- store all outputs into the SCEM archive

Every build must generate a complete, reproducible evidence package.

15.12 TCA: Toolchain Confidence Assessment

CRSS adopts a formal TCA process inspired by DO-330 and ISO 26262-8.
TCA Requirements:

- classify tools by their impact on safety
- verify correctness of safety-impacting tools
- validate tool configurations
- ensure determinism of tool behavior
- version-lock all tools
- maintain tool usage evidence
- require re-assessment on any tool upgrade

TCA ensures that tools used to validate safety cannot introduce errors.

15.13 Permitted vs Forbidden Automation

Permitted:

- CI/CD pipelines
- static analyzers
- coverage tools
- deterministic test runners
- reproducible build tooling

Forbidden:

- auto-formatters that change semantics
- auto-refactoring tools
- dynamic dependency upgraders
- tools that modify code silently
- any automation that introduces nondeterminism

Automation must never obscure program behavior.

15.14 Tool Qualification Strategy

Tool qualification steps:

1. Define tool purpose
2. Classify tool impact on safety
3. Document tool configuration
4. Validate expected behavior 5.

Record tests and evidence

6. Lock tool version in CBM
7. Periodically re-validate

Tools critical to safety verification must meet stricter qualification requirements.

15.15 Automation Review Checklist

Before certification, the project must validate:

- static analysis fully configured
- CI/CD pipelines deterministic
- no mutable scripts in build chain
- TCA evidence complete
- reproducible build verified
- test automation validated

Failures here block release certification.

15.16 Summary of Tooling & TCA Requirements

CRSS defines a rigorous and certifiable tooling ecosystem. Static analysis, reproducible builds, deterministic automation, evidence capture, and TCA form the foundation that allows Python systems to reach ASIL-D and SIL-3 supervisory readiness.

16 Safety-Level Mapping & Certification Readiness

16.7 Purpose of this Section

This section defines how CRSS-Python aligns with established industry certification levels and how it provides a defensible, evidence-driven basis for supervisory safety certification in domains such as automotive (ISO 26262), industrial automation (IEC 61508), medical devices (IEC 62304), and avionics (DO-278A).

16.8 Mapping to ISO 26262 (ASIL A–D)

CRSS-Python supports:

- ASIL A/B: fully supported with Core or Strict
- ASIL C: Strict with Safety Level B or C
- ASIL D: Strict-Level-A (supervisory only)

CRSS cannot be used for primary actuation loops (e.g., torque control, braking commands) but is fully appropriate for supervisory, advisory, fusion, and monitoring components.

16.9 Mapping to IEC 61508 (SIL 1–4)

CRSS supports:

- SIL 1–2: fully supported
- SIL 3: supported for supervisory logic
- SIL 4: not applicable for primary safety mechanisms

SIL 3 supervisory systems are achievable due to deterministic execution guarantees within Strict-Level-A.

16.10 Mapping to DO-178C / DO-278A

CRSS supports:

- DO-278A: ground systems
- DO-178C DAL D/C non-flight-critical utilities
- DAL B/A: not applicable for airborne primary functions

Python is not accepted for flight control or primary airborne safety functions, but CRSS enables high-integrity ground and support systems.

16.11 Mapping to IEC 62304 (Medical Class A–C)

CRSS supports:

- Class A/B: fully supported
- Class C: Strict-Level-A possible for supervisory modules Class C primary therapy-control loops remain unsuitable for Python.

16.12 What CRSS CAN Certify

CRSS enables certification of:

- supervisory safety functions
- fusion and decision-support algorithms
- safety monitors and watchdog logic
- diagnostics and health management
- control advisory modules
- high-integrity analytics
- safety-relevant edge services

16.13 What CRSS CANNOT Certify

CRSS cannot certify:

- primary real-time actuation loops
- hard real-time constraints
- single-point-of-failure safety channels
- systems requiring deterministic WCET at microsecond level

16.14 Safety Argument Structure

CRSS provides a multi-layered safety case:

- deterministic critical execution
- strict phase separation
- safety-level propagation rules
- isolation of nondeterministic behavior
- verifiable memory, GC, and timing constraints
- reproducible deployment environment
- SCEM evidence artifacts
- TCA-supported toolchain reliability
- end-to-end lifecycle governance

16.15 Evidence for Certification

Certification bodies will evaluate:

- rule compliance (GCS / RCR)
- CBM (Baseline Manifest)
- SCEM (evidence package)
- MAR (mode assignment register)
- TEP (Test Evidence)

- TCA (Toolchain confidence)
- configuration integrity
- reproducible build proof
- determinism verification
- safety-level assignment rationale

CRSS provides all required artifacts in predictable, inspectable form.

16.16Limitations & Constraints

CRSS's limitations are clear:

- Python remains non–hard-real-time
- interpreter nondeterminism cannot be removed entirely
- certified execution depends on strict architectural constraints

These limitations are explicitly documented to maintain regulator confidence.

16.17Summary of Certification Applicability

CRSS-Python provides a certifiable supervisory-safety framework capable of achieving:

- ASIL D (supervisory)
- SIL 3 (supervisory)
- DO-278A high integrity
- IEC 62304 Class C (supporting)

This represents one of the most advanced and certifiable Python safety ecosystems available.

17 Conclusion & Future Directions

CRSS-Python v1.0.0 represents the most comprehensive, certifiable, and lifecycle-complete safety framework ever developed for Python. It bridges the gap between high-level language flexibility and the deterministic execution, structural integrity, and governance rigor required for modern safety-critical systems.

The future of CRSS-Python includes:

- Domain-specific annexes (automotive, medical, industrial automation)
- Reference implementations of certified-grade architectures
- A public rule catalog and compliance validator
- A standardized test suite for determinism and robustness
- Formal semantics for critical-phase execution

CRSS does not attempt to make Python a hard-real-time language; instead, it defines a safe and certifiable subset for supervisory safety logic, aligned with ASIL D, SIL-3, IEC 62304 Class C, and DO-278A high-integrity ground systems.

The ongoing goal is to evolve CRSS as a living standard: stable, transparent, rigorously reviewed, and aligned with real-world certification expectations.

18 Glossary

Critical Phase: Deterministic, allocation-free, I/O-free execution path with GC disabled.

Non-Critical Phase: Operational phase where I/O and allocation are allowed but Strict rules remain enforced.

CBM (Configuration Baseline Manifest): Frozen snapshot of code, interpreter, OS, dependencies, tools, and infrastructure required to reproduce the certified build.

SCEM (Safety & Compliance Evidence Model): collecting all verification and lifecycle artifacts.

MAR (Mode Assignment Register): Defines Safety Levels A/B/C for units and establishes criticality mapping.

RCR (Rule Compliance Report): complete report of rule compliance exercise

Strict-Level-A: Highest safety mode; no MUST-NOT violations anywhere; deterministic critical paths.

TCA (Toolchain Confidence Assessment): Formal evaluation of tools used for verification and build processes.

Determinism: Guarantee that critical execution always behaves identically given the same inputs.

18. Bibliography & References

- ISO 26262: Road Vehicles - Functional Safety
- IEC 61508: Functional Safety of Electrical/Electronic Systems
- IEC 62304: Medical Device Software
- DO-178C & DO-278A: Software Considerations in Airborne/Ground-Based Systems

19. About CRSS & Licensing

CRSS: Certifiable Reliability Safety Standard for Python defines a deterministic, certifiable, and structurally safe subset of the Python language combined with complete lifecycle governance, deployment integrity, configuration control, and formal verification requirements.